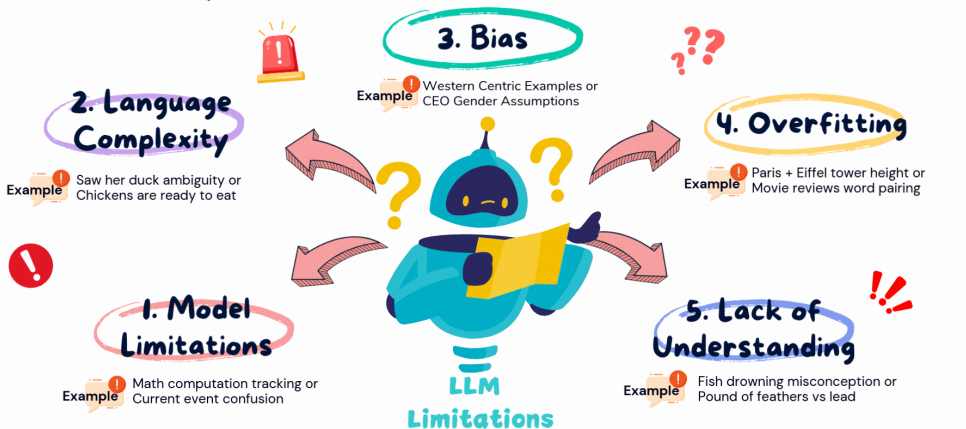


Hallucination in RAG & Agents

Why it happens, how to catch it, and every technique that actually reduces it.

Why LLM Hallucinate?



📄 ✕ 🌐 @pvergadia 📺 thecloudgirl



AI Coach John



FOLLOW

Repost

What Is Hallucination, Exactly?

Not all wrong answers are the same failure

Core Definition

The model generates content that is fluent and confident, but not actually supported by reality or by the provided context.

Intrinsic

Output directly contradicts the source or context provided (context says X, model says not-X).

Extrinsic

Output adds invented information not present in the source or context at all.

Factual

The claim is objectively false in real-world reality.

Faithfulness

Not supported by retrieved context.
Primary metric for RAG systems.



Why grounding matters: In RAG, even an objectively true statement is considered a faithfulness failure if the retrieved document did not state it.



AI Coach John



FOLLOW

 Repost



ROOT CAUSES

Why LLMs Hallucinate At All

Root causes baked into how LLMs work



Training Objective

LLMs are trained to predict plausible next tokens, not to verify truth. Fluency and confidence are optimized for, not accuracy.



No Built-In Fact-Checking

The model has no internal mechanism to look something up unless you explicitly give it external tools or RAG context.



Parametric Knowledge Gaps

Training data has cutoffs and gaps. When asked about unknown topics, models generate plausible-sounding guesses instead of admitting uncertainty.



Overconfidence by Default

Nothing in standard decoding forces the model to express doubt. Wrong answers and right answers look equally confident.



Structural reality: You cannot fully eliminate hallucination. The goal is systematic detection + reduction.



AI Coach John

 **PROITBRIDGE**
Bridging the AI Divide

FOLLOW

 **Repost**

Why RAG Doesn't Fully Solve It

RAG helps, but does not eliminate hallucination

Query



Retrieval Leak



Generation



Output

⚠️ Retrieval Failure

Nothing relevant was retrieved, but the model answers anyway using memorized parametric knowledge instead of saying "I don't know."

🔍 Context Ignoring

Model receives good context but relies on internal priors, especially when context conflicts with what it "remembers."

🔵 Partial Grounding

Part of the answer is grounded in context, part is invented. Much harder to catch than fully fabricated text.

⚡ Conflicting Sources

Retrieved chunks disagree with each other; the model blends them without flagging the conflict.



Why Agents Make It Worse

Multi-step systems compound hallucination risk

Step 1 Error



Tool Call



Poisoned Chain

Error Propagation

If step 1 hallucinates a fact, every later step reasoning from it inherits the error. One false premise corrupts the entire chain.

Tool Call Hallucination

Agents can fabricate tool outputs, invent non-existent parameters, or claim a tool succeeded when it failed.

Hidden Intermediate Steps

Sub-tasks rarely get the same scrutiny as final answers, hiding hallucinations in the reasoning trace.

Multi-Hop Drift

In multi-hop retrieval, an early sub-question error feeds into the next search query, drifting further each hop.



Types of Hallucination In RAG

What hallucination looks like in a RAG system concretely

Unsupported Claim

A statement with no matching content in any retrieved chunk.

Fabricated Citation

Citing "[1]" or "according to the document" for a claim that source never actually stated.

Context Contradiction

Stating something that directly conflicts with the retrieved text.

Over-Generalization

Source states a narrow or conditional rule; model states it as a broad universal fact.

Source Conflation

Blending facts from two different chunks into one claim that neither chunk supports on its own.



How to Identify: The Manual Way

Start here: the low-tech but essential first audit

1. Side-by-Side Reading

Read retrieved context and generated answer side by side. Verify that every single claim maps directly to text in the context.

2. Verify Every Citation

Check each cited reference tag ([1], [2]). Does that specific chunk actually support the attributed sentence?

3. Watch Suspicious Specifics

Audit exact numbers, dates, and proper names. Models frequently invent precise-sounding numerical details.

4. The Memory Test

Ask: if I removed retrieved context entirely, could the model have produced this exact answer from memory? If yes, check grounding.

→ **Next step:** Once you know what to look for manually, automate these checks into programmatic gates.



Faithfulness Scoring (NLI-Based)

Automatically checking if the answer is entailed by context

```
from transformers import pipeline

nli_model = pipeline("text-classification",
                     model="microsoft/deberta-large-mnli")

def check_faithfulness(context: str, claim: str):
    result = nli_model(f"{context} [SEP] {claim}")
    return result[0]["label"] # ENTAILMENT / NEUTRAL / CONTRADICTION

label = check_faithfulness(
    context="Refund window is 30 days for physical products.",
    claim="Customers can return any product within 30 days."
)
print(label) # "NEUTRAL" (overgeneralized beyond physical)
```

✓ How NLI Entailment Works

NLI models judge whether premise text logically entails a hypothesis sentence. Perfect fit for RAG grounding verification.



Best practice: Score claim-by-claim (sentence level), not full paragraphs at once, to prevent ungrounded sentences from blending in.



AI Coach John



FOLLOW

 Repost

Self-Consistency Checking

If the model cannot repeat itself, it is probably guessing

```
def self_consistency_check(prompt, llm, n=5):
    answers = [llm.invoke(prompt, temperature=0.7).content
                for _ in range(n)]

    from collections import Counter
    normalized = [a.strip().lower()[:100] for a in answers]
    counts = Counter(normalized)
    most_common_count = counts.most_common(1)[0][1]

    return most_common_count / n, answers

ratio, answers = self_consistency_check("Refund policy?", llm)
print(f"Consistency: {ratio:.0%}") # Low ratio -> review flag
```

Why Variance Signals Hallucination

Grounded facts remain stable across stochastic sampling. Invented details fluctuate across repeated generations.



Production tip: Reserve multi-sample consistency checks for high-stakes audits or spot-check evaluation pipelines.



Citation & Source Verification

Verify every citation actually says what it is cited for

```
import re

def verify_citations(answer: str, retrieved_chunks: dict):
    claims = re.split(r'(<=|.!?)\s+', answer)
    flagged = []
    for claim in claims:
        match = re.search(r"\\(\\d+)\\\"", claim)
        if match:
            source_id = f"[{match.group(1)}]"
            source_text = retrieved_chunks.get(source_id, "")
            if check_faithfulness(source_text, claim) != "ENTAILMENT":
                flagged.append((claim, source_id))
    return flagged
```

Catching Fabricated Citations

Programmatically extracts citation brackets and runs NLI entailment against the specific referenced chunk.



Why combine: Checking citation existence is not enough. You must verify that the cited chunk actually entails the claim.



AI Coach John



FOLLOW

 Repost

Token-Level Confidence

Using the model's own log-probability signals

```
import torch

def get_token_confidence(model, tokenizer, prompt, answer):
    inputs = tokenizer(prompt + answer, return_tensors="pt")
    with torch.no_grad():
        outputs = model(**inputs, labels=inputs["input_ids"])
        probs = torch.softmax(outputs.logits, dim=-1)
        token_confidences = probs.max(dim=-1).values
    return token_confidences.mean().item()

conf = get_token_confidence(model, tokenizer, prompt, answer)
if conf < 0.5:
    flag_for_review(answer)
```

Entity-Specific Probability Drops

Low log-probabilities on factual tokens (names, dates, numerical values) strongly indicate ungrounded guessing.



Requirement: Requires access to token logprobs (open-source models hosted via vLLM or APIs exposing logprobs).



AI Coach John



FOLLOW

 Repost

LLM-as-Judge Fact-Checking

Using a second LLM call to grade the first one

```
judge_prompt = """You are a strict fact-checker. Given context and  
answer below, list claims NOT directly supported by context.  
If all claims are supported, respond "NONE".
```

```
Context: {context}  
Answer: {answer}  
Unsupported claims:"""
```

```
def llm_judge(context, answer, judge_llm):  
    return judge_llm.invoke(  
        judge_prompt.format(context=context, answer=answer)  
    ).content
```

Flexible Nuanced Grading

Catches subtle over-generalizations and multi-sentence conflation that smaller NLI classifiers sometimes overlook.



Best practice: Use a stronger or independent model as judge. Grading your own output with the same model risks blind spots.





REDUCTION TECHNIQUE 01

Prompt-Level Grounding

The cheapest fix: tell the model explicitly what to do

```
grounded_prompt = """Answer the question using ONLY the information  
in the context below. Do not use any outside knowledge.
```

```
If the context does not contain enough information to answer,  
respond exactly: "I don't have enough information to answer this."
```

```
Context:  
{context}
```

```
Question: {question}  
Answer: """
```



Permission to Refuse

Giving explicit permission to say "I don't know" removes the conversational pressure that drives models to invent answers.



Highest ROI: Zero cost, zero extra latency. Always implement strict refusal instructions as your baseline defense.



AI Coach John



FOLLOW

 Repost

Better Retrieval Quality

Garbage retrieval in, hallucinated answer out

✂ 1. Semantic Chunking

Split by semantic boundaries rather than arbitrary character counts to keep complete thoughts intact.

🔗 2. Hybrid Search (BM25 + Dense)

Combine keyword BM25 with dense embeddings to catch exact product IDs and technical terms.

✍ 3. Query Rewriting

Reformulate vague user queries into optimized retrieval prompts before searching the index.

⬇️ 4. Top-K + Cross-Encoder Rerank

Retrieve 20 candidates fast, then use a Cross-Encoder to rank the top 5 highest-precision chunks.



Root cause fix: Most RAG hallucination is retrieval failure wearing a prompt problem disguise. Audit retrieval first.



AI Coach John

 PROITBRIDGE
Bridging the AI Gap for Every Team

FOLLOW

 Repost

Context Grading Before Gen

Do not generate from context you have not verified is useful

```
grade_prompt = """Do these documents contain enough relevant
information to answer the question? Answer only "YES" or "NO".
```

```
Question: {question}
Documents: {documents}
Answer: """
```

```
def grade_context(question, docs, llm):
    res = llm.invoke(grade_prompt.format(question=question,
                                         documents=docs))
    return "YES" in res.content

if not grade_context(question, retrieved_docs, llm):
    retrieved_docs = fallback_retrieve(question) # Corrective RAG
```

🛡️ Corrective RAG Pattern (CRAG)

Intercepts weak retrieval before generation starts. Triggers web search fallback or clean refusal instead of forcing bad output.

- ✓ **Why it works:** Prevents the model from trying to connect non-existent dots when retrieved docs miss the target.



Context Filtering & Compression

Less but more relevant context reduces confusion

```
from sentence_transformers import CrossEncoder

relevance_model = CrossEncoder("cross-encoder/ms-marco-MiniLM-L-6-v2")

def filter_context(question, chunk_text, threshold=0.3):
    sentences = chunk_text.split(". ")
    scores = relevance_model.predict([(question, s)
                                     for s in sentences])
    relevant = [s for s, score in zip(sentences, scores)
               if score > threshold]
    return ". ".join(relevant)
```



Stripping Tangential Noise

Filters out irrelevant sentences within retrieved chunks before passing text into the final prompt window.



Precision benefit: Shorter, cleaner context reduces attention drift and prevents models from latching onto tangential details.



AI Coach John



FOLLOW

 Repost



REDUCTION TECHNIQUE 05

Decoding Parameters

Simple generation settings that reduce invented detail

```
# Low temperature + tight nucleus sampling for factual QA
response = llm.invoke(
    prompt,
    temperature=0.1, # Minimizes stochastic wandering
    top_p=0.9        # Truncates low-probability tail tokens
)
```



Lower Temperature (0.0 to 0.2)

Keeps the model focused on high-probability, factual continuations grounded in the immediate prompt text.



Structured Decoding Constraints

Use schema enforcers like Pydantic, Instructor, or Outlines so models cannot wander into ungrounded narrative prose.



Easy win: Never use high temperature (0.7+) for factual enterprise RAG tasks. Always tighten decoding first.



AI Coach John



FOLLOW

 Repost

Self-RAG Reflection Loop

Making the model critique its own output before returning

```
reflect_prompt = """Does the answer below rely ONLY on context,  
with no invented facts? Answer "SUPPORTED" or "UNSUPPORTED".  
Context: {context} | Answer: {answer} | Verdict:"""  
  
def self_rag_loop(question, context, llm, max_retries=2):  
    answer = llm.invoke(gen_prompt.format(context=context,  
                                          question=question)).content  
  
    for _ in range(max_retries):  
        verdict = llm.invoke(reflect_prompt.format(  
            context=context, answer=answer)).content  
        if "SUPPORTED" in verdict:  
            return answer  
        answer = llm.invoke(gen_prompt.format(context=context,  
            question=question) + "\nBe strictly conservative.")).content  
    return answer
```

Self-Correction Before User Sees Output

Draft answers are critiqued against context. If unsupported claims are found, the model revises conservatively.



Always cap retries: Limit reflection loops to 2 retries maximum to prevent infinite latency loops on difficult queries.



AI Coach John



FOLLOW

 Repost

Agentic Self-Correction Loop

Full grade -> retry -> correct loop as a state graph

```
from langgraph.graph import StateGraph, END

graph = StateGraph(dict)
graph.add_node("grade_context", grade_context_node)
graph.add_node("generate", generate_node)
graph.add_node("reflect", reflect_node)
graph.add_node("correct_retrieval", correct_retrieval_node)

graph.set_entry_point("grade_context")
graph.add_conditional_edges("grade_context",
    lambda s: "generate" if s["sufficient"] else "correct_retrieval",
    {"generate": "generate", "correct_retrieval": "correct_retrieval"})
graph.add_edge("correct_retrieval", "generate")
graph.add_edge("generate", "reflect")
graph.add_conditional_edges("reflect",
    lambda s: END if s["supported"] else "generate",
    {END: END, "generate": "generate"})
```

Multi-Point Safeguards

Verifies both retrieval sufficiency and generation grounding in a unified LangGraph workflow.



High reliability: Ideal for mission-critical queries where accuracy outweighs extra LLM call latency.



AI Coach John



FOLLOW

 Repost

Fine-Tuning for Faithfulness

When prompting is not enough, train grounding in

1. Supervised Fine-Tuning (SFT)

Train models on grounded QA pairs where every answer strictly quotes context, including explicit refusals when context is missing.

2. DPO / RLHF Preference Pairs

Use Direct Preference Optimization comparing faithful answers against hallucinated drafts to penalize ungrounded completions.

When Worth It

High-stakes regulated domains (medical, legal, financial) where prompt-level instructions need structural weight reinforcement.

When Not Needed

General enterprise RAG. Exhaust prompt engineering, retrieval tuning, and Corrective RAG first.



Post-Gen Fact-Checking

A dedicated verification step before the answer ships

```
def generate_with_verification(question, context, llm):
    answer = generate_node({"question": question,
                           "context": context})["answer"]
    faithfulness_label = check_faithfulness(context, answer)

    if faithfulness_label != "ENTAILMENT":
        return ("Note: I could not fully verify every detail "
               "against the source text: " + answer)
    return answer
```

Active Interception Gate

Turns NLI faithfulness detection into an active production firewall rather than passive background logging.



UX Best practice: A transparent verification disclaimer is often better user experience than silently blocking responses.



AI Coach John



FOLLOW

 Repost



REDUCTION TECHNIQUE 10

Multi-Agent Verification

Have a second independent agent check the first work

```
gen_answer = generator_agent.invoke({"question": q,  
                                     "context": ctx})  
  
verifier_prompt = """"You are an adversarial fact-checker. Your  
ONLY job is to find flaws. Given context and answer below, list  
every claim that lacks direct context support, however small.""">  
  
verifier_result = verifier_agent.invoke(  
    verifier_prompt + f"\nContext: {ctx}\nAnswer: {gen_answer}"  
)
```



Adversarial Framing Works Better

Telling a verifier agent to "find flaws" produces significantly stricter scrutiny than asking "is this correct?"



Independent models: Use a different model family for the verifier to eliminate shared architectural blind spots.



AI Coach John



FOLLOW

 Repost

Agent Hallucination: Tools

Agents can hallucinate actions, not just facts

Fake Results

Claiming a tool returned success when it was never executed.

Invented Args

Passing made-up arguments not grounded in context.

```
@tool
def send_email(to: str, body: str) -> str:
    """Send email. Returns explicit confirmation or error."""
    try:
        res = email_client.send(to, body)
        return f"CONFIRMED: sent to {to}, id={res.id}"
    except Exception as e:
        return f"FAILED: {str(e)}"
```

Require Literal Output Quoting

Force tools to return explicit confirmation strings and require agents to quote literal return values in final responses.





Cost vs Rigor: Choosing Fixes

You do not need every technique on every request

TECHNIQUE	EXTRA CALLS	BEST FOR
Prompt Grounding	0 calls	Every app baseline
Better Retrieval	0 runtime	All RAG systems
Decoding Params	0 calls	All factual QA
Context Grading	+1 call	Medium-stakes RAG
Self-RAG Loop	+1 to 2	Medium-to-high stakes
Agentic Self-Correct	+2 to 4	High-stakes queries
LLM Judge Gate	+1 call	Compliance workflows
Multi-Agent Verify	+2+ calls	Legal, Medical, FinTech
Fine-Tuning (SFT)	0 runtime	Sustained enterprise QA



Evaluation Metrics & Benchmarks

Measuring hallucination rate over time, not just once

```
from ragas import evaluate
from ragas.metrics import faithfulness, context_precision

scores = evaluate(eval_dataset,
                  metrics=[faithfulness, context_precision])
```

✓ 1. Faithfulness Score (RAGAS / NLI)

Percentage of generated claims logically entailed by retrieved text.

📈 2. Live Hallucination Rate

Proportion of production traffic flagged by automated evaluators.

🏆 TruthfulQA

Tests factual misconception resistance.

📋 HaluEval

QA & summarization hallucination benchmark.



Production Monitoring **At Scale**

Catching drift in hallucination rate before users do

1. Sample Live Traffic Scoring

Run automated NLI faithfulness scoring on a 5% to 10% sample of live production traffic to establish trend lines affordably.

2. Threshold Alerts

Trigger automated alerts when faithfulness drops. Catch retrieval degradation or model prompt drift immediately.

3. Log Full Audit Traces

Store prompt, retrieved chunks, output, and judge verdicts in LangSmith or Langfuse for every flagged incident.

4. Human Review Queue

Route flagged edge cases to human reviewers to continuously expand evaluation golden datasets.



Common Pitfalls To Avoid

Mistakes that undermine otherwise solid fixes

1. Only Tuning Prompts, Ignoring Retrieval

Most hallucination is retrieval quality failure wearing a prompt engineering disguise.

2. No Retry Caps on Self-Correction

Stuck reflection loops can iterate infinitely without a hard max-retries ceiling.

3. Same-Model Self-Grading

Grading output with the exact same model that generated it risks shared architectural blind spots.

4. Treating Hallucination as Binary

Partial grounding is common. Always measure claim-level faithfulness rather than full-answer pass/fail.

5. Zero Production Monitoring

Testing once at launch is not enough. Documents, queries, and model endpoints drift over time.

